Детекция рейсов карьерных самосвалов методом шаблонной векторизации: архитектура, алгоритмы и инженерный анализ

Введение

В горнодобывающей промышленности точный учёт рейсов карьерных самосвалов — ключевой фактор управления производительностью. Традиционные системы диспетчеризации (DISPATCH от Modular Mining, Wenco, российская «Карьер») опираются на GPS-зоны: система фиксирует въезд самосвала в зону экскаватора или пункта разгрузки и по факту пересечения геозон формирует рейс. Однако этот подход не улавливает аномалии внутри цикла — простои, заторы, сбои датчиков, затянувшиеся обеды — и не позволяет классифицировать тип рейса по его «форме».[1][2]

Альтернативный подход реализован в системе **«Симуляция и детекция рейсов»** — серверно-клиентском приложении (Go + React + PostgreSQL), которое распознаёт рейсы в реальном времени путём **шаблонной векторизации** телеметрических данных скорости и веса. Вместо привязки к координатам система строит вектор из скользящего окна телеметрии и сравнивает его с заранее сохранёнными эталонными шаблонами через меру сходства (косинусное сходство или нормы L1/L2). Когда степень совпадения превышает порог — рейс считается обнаруженным.[3][4]

Данная статья детально разбирает архитектуру, алгоритмическую основу и инженерные решения этого подхода,

сопоставляя его с мировой практикой детекции рабочих циклов тяжёлой техники.

Контекст проблемы: фазовая модель рейса самосвала

Рабочий цикл карьерного самосвала — это строго последовательная цепочка технологических операций, повторяющаяся десятки раз за смену:[5][6]

Фаза	Скорость	Вес	Типичная длительность
Погрузка	≤ Vmin (≈0.5 км/ч)	Линейно нарастает от 0 до M	~120 сек
Транспортировка (гружёный)	Vmin…Vmax (до 40 км/ч)	Mmin…Mmax (~90–100 т) + шум	~300 сек
Разгрузка	Падает до Vmin	Падает от M до 0	~60 сек
Возврат (порожний)	Vmin…Vmax	≈ Мпорожний (≈1 т) + шум	~240 сек

Эти данные заложены в генератор симуляционного потока приложения. Каждая фаза характеризуется характерным «профилем» пары скорость–вес: при погрузке скорость минимальна, а вес растёт; при транспортировке скорость высокая, вес стабильный; при разгрузке оба параметра падают. Этот двумерный профиль рейса — главный сигнал для распознавания.[4]

В реальных условиях на профиль накладывается шум от неровности дороги, вибрации подвески и погрешностей датчиков. Система моделирует это через три параметра: шум скорости (±0.5 км/ч), шум веса (±1 т), шум веса при погрузке из-за

амортизаторов (±2 т). Дополнительно, длительность каждой фазы может отклоняться от эталона на настраиваемый процент.[4]

Архитектура системы

Стек и структура

Система построена на стеке **Go 1.22+ / Gin / pgx** на бэкенде и **React / ECharts / WebSocket** на фронтенде, с **PostgreSQL** в качестве хранилища. Архитектура разделена на модули с чёткими зонами ответственности:[7]

```
cmd/server/          → Точка входа
internal/
  ├── config/        → Конфигурация (env + Viper)
  ├── db/            → Миграции, подключение
  ├── domain/        → Модели домена
  ├── repository/    → Доступ к БД
  ├── service/
  │   ├── generator/  → Генератор данных по фазам + шум
  │   ├── queue/      → Потокобезопасная очередь данных
  │   ├── vector/     → Векторизация и нормализация
  │   └── recognition/→ Скользящее окно + сравнение
  ├── api/           → HTTP handlers + WebSocket hub
  └── worker/        → Фоновые воркеры
web/                 → React SPA
```

Потоки данных

Архитектура организует три конвейерных потока, связанных через каналы Go (goroutines + channels):[7]

1. **Генерация.** Сервис `generator` в отдельной горутине по таймеру (каждые R секунд, по умолчанию 10) вычисляет скорость и вес по текущей фазе рейса, накладывает гауссов/равномерный шум и отправляет точку в канал.

2. **Очередь.** Сервис `queue` читает из канала, буферизует (последние P минут), отдаёт данные подписчикам WebSocket и в сервис распознавания. Буфер ограничен по времени — это sliding buffer, из которого удаляются устаревшие точки.
3. **Распознавание.** Сервис `recognition` накапливает скользящее окно (размер определяется шаблонами), строит вектор, нормализует, сравнивает с шаблонами из БД. При совпадении ≥ порога — рейс найден, сохраняется в PostgreSQL, событие уходит по WebSocket.

Транспортный уровень

Система использует **REST API** для управления (старт/стоп генерации, настройки, CRUD шаблонов и рейсов) и **WebSocket** для стриминга данных в реальном времени. Через WebSocket клиент получает два типа сообщений:[8]

- `point` — новая точка телеметрии (timestamp, speed, weight, phase)
- `trip_found` — событие обнаружения рейса (UUID, интервал, шаблон, процент совпадения)

Ядро алгоритма: шаблонная векторизация

Принцип работы

Центральная идея системы — представление отрезка телеметрии (скорость + вес за период) в виде **единого числового вектора** и сравнение этого вектора с эталонными шаблонами через метрику сходства. Этот подход близок к тому, что в литературе по анализу временных рядов называется **template matching** или **vector-based similarity search**.[9][10][7]

Алгоритм работает по следующей схеме:

1. **Сбор шаблона.** Оператор на странице «История» выделяет участок телеметрии, характерный для определённого типа рейса (например, «нормальный рейс», «затор у экскаватора», «рейс с перекуром»). Система сохраняет

массивы скоростей и весов этого участка, вычисляет из них нормализованный вектор и сохраняет в таблицу trip_template_vectors.[7]
2. **Скользящее окно (sliding window).** При поступлении каждой новой точки телеметрии система поддерживает буфер последних N значений скорости и M значений веса, где N и M определяются размерами шаблонов (от меньшего к большему). Окно сдвигается с каждой новой точкой — это классическая техника sliding window, широко применяемая в потоковой обработке данных.[11][12][3]
3. **Векторизация.** Содержимое окна конкатенируется в один вектор: например, 100 значений скорости + 95 значений веса = 195 компонент. Каждый компонент нормализуется в процентах (0..1 или 0..100).[7]
4. **Сравнение.** Нормализованный вектор текущего окна сравнивается со всеми шаблонами через **косинусное сходство** (или норму L1/L2). Если лучшее совпадение ≥ порога (по умолчанию 85%) — рейс считается обнаруженным.[4][7]
5. **Дедупликация.** После обнаружения рейса устанавливается «период охлаждения» (cooldown_after_trip_sec) — новый рейс не может начаться раньше конца предыдущего плюс это число секунд. Рейсы не пересекаются по интервалам в любом случае.[3]

Косинусное сходство: почему оно подходит

Косинусное сходство измеряет угол между двумя векторами, игнорируя их абсолютную длину. Формула:[13][14]

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Для предварительно нормализованных векторов (что делает система) это сводится к простому скалярному произведению. Это даёт три ключевых преимущества для задачи детекции рейсов:[15]

- **Инвариантность к масштабу.** Если абсолютные значения скорости/веса сместились (другой самосвал, другая

загрузка), но «форма» профиля та же — сходство останется высоким.
- **Вычислительная эффективность.** Скалярное произведение двух векторов из ~200 компонент — наносекунды, что критично для реального времени.
- **Интерпретируемость.** Результат от 0 до 100% — понятная метрика для диспетчера.

В академической литературе отмечается, что для задач сравнения формы временных рядов косинусное сходство и евклидово расстояние тесно связаны и оба эффективны. При этом для задач, где важна именно форма сигнала (а не абсолютные значения), косинусное сходство предпочтительнее.[16][17]

Сравнение с альтернативными подходами

Метод	Инвариантность к сдвигу по времени	Вычислительная сложность	Интерпретируемость	Реальное время
Косинусное сходство + sliding window (данная система)	Нет (фиксированное окно)	O(n·k), n — шаблоны, k — размер вектора	Высокая (% совпадения)	Да
DTW (Dynamic Time Warping)	Да	$O(n \cdot m^2)$, m — длина серии	Средняя	Ограниченно[18]
Bi-LSTM + Random Forest	Да	Высокая (обучение)	Низкая (чёрный ящик)	Ограниченно[19]
GPS-зоны	Неприменимо	O(1)	Высокая	Да
GMM-кластеризация	Частично	Средняя	Средняя	Частично[20]

Подход, выбранный в системе, занимает уникальную нишу: он проще и быстрее DTW, не требует обучения модели (в отличие от нейросетевых методов), но при этом способен различать разные типы рейсов — чего не может GPS-зонирование.

Система шаблонов: от «нормального рейса» до аномалий

Одно из ключевых инженерных решений — **библиотека шаблонов**, покрывающая не только штатные, но и нештатные ситуации. В системе сохранено 19 шаблонов, которые можно классифицировать:[21]

Штатные шаблоны

- **«Основной»** (серия 1–6) — эталонный нормальный рейс с небольшими вариациями. Совпадения с этими шаблонами показывают 93–97%.[22]
- **«Запасной»** — альтернативный маршрут с другим профилем скоростей.

Шаблоны с отклонениями

- **«Затор у экскаватора»** — характерный паттерн: длительная стоянка с нулевой скоростью и нарастающим весом (медленная загрузка в очереди).[22]
- **«Рывок»** — резкие ускорения/торможения в транспортной фазе.
- **«Перекур»** / **«Обед груженым»** / **«Долгий обед»** / **«Заправка и обед»** — остановки посреди цикла с характерными паузами в скоростном профиле.

Аварийные шаблоны

- **«Сбой датчиков подвески»** — аномальный шум в весовых данных при нормальной скорости.
- **«Ошибка при погрузке»** — прерванная загрузка с характерным «зубчатым» профилем веса.
- **«Ремонт»** / **«Ремонт груженным»** — длительный простой с сохранением или без груза.
- **«Аномалия»** — неклассифицируемые отклонения.

Такой подход по сути реализует **кодификацию экспертного знания** диспетчеров и машинистов в машиночитаемые шаблоны. Это ближе к концепции «шейплетов» (shapelets) в

анализе временных рядов — характерных подпоследовательностей, которые определяют принадлежность к классу.[23]

Интерфейс и рабочие процессы

Страница «Симуляция»

Центральная страница оператора, объединяющая управление, мониторинг и аналитику:[3]

- **Управление:** кнопки Старт/Стоп/Очистить с подтверждающими модальными окнами; чекбокс включения распознавания; индикатор статуса с анимацией.
- **График реального времени:** ECharts-график скорости и веса за последние 30/60/90/120 минут с ползунком масштаба. Опциональная подсветка анализируемого sliding window.
- **Блок «Анализ»:** количество загруженных шаблонов, накопленных точек, вычислен ли вектор, время расчёта (мс), лучшее совпадение с указанием шаблона и процента, список сравнений по всем шаблонам.
- **Таблица рейсов:** последние обнаруженные рейсы (по WebSocket) с начальным/конечным временем, порогом, процентом совпадения и именем шаблона.

Страница «История»

Инструмент для ретроспективного анализа и создания новых шаблонов:[3]

- Выбор временного интервала и загрузка исторических данных.
- График с прямоугольниками найденных рейсов (подпись: имя шаблона или «не найден»).
- Форма сохранения выделенного участка как нового шаблона.
- **Перерасчёт рейсов** — пересчёт обнаруженных рейсов за период с новыми шаблонами или порогами (фоновый процесс с прогрессом).

Страница «Шаблоны»

Управление библиотекой шаблонов: поиск по имени и длительности, просмотр графиков скоростей/весов шаблона, редактирование (обрезка диапазона ползунком), удаление.[3]

Математическая модель распознавания

Формализация

Пусть шаблон T состоит из массива скоростей $s_1^T, \ldots, s_n^T$ и весов $w_1^T, \ldots, w_m^T$. Вектор шаблона:

$$\vec{T} = \left[\hat{s}_1^T, \ldots, \hat{s}_n^T, \hat{w}_1^T, \ldots, \hat{w}_m^T\right]$$

где $\hat{s}_i^T = s_i^T / \max(s^T)$ — нормализация в проценты.[7]

Аналогично, для текущего скользящего окна W:

$$\vec{W} = \left[\hat{s}_1^W, \ldots, \hat{s}_n^W, \hat{w}_1^W, \ldots, \hat{w}_m^W\right]$$

Мера совпадения:

$$\mathrm{match}(\vec{W}, \vec{T}) = \frac{\vec{W} \cdot \vec{T}}{\|\vec{W}\| \cdot \|\vec{T}\|} \times 100\%$$

Рейс обнаружен, если $\mathrm{match} \geq \theta$, где θ — порог (по умолчанию 85%).[4]

Дополнительные фильтры

Система поддерживает проверку «у оси»: рейс сохраняется только если в начале и конце скользящего окна скорость и вес не превышают заданных порогов (рекомендуется 5 км/ч и 15–20 т). Это отсекает ложные срабатывания, когда самосвал находится посреди транспортировки.[3]

Преимущества подхода

По сравнению с GPS-зонированием

GPS-системы определяют рейс по факту пересечения геозон — погрузочной и разгрузочной. Это надёжно для учёта количества рейсов, но не позволяет:[2][1]

- Классифицировать тип рейса (нормальный vs. с аномалией).
- Работать без GPS (внутри тоннелей, при потере сигнала).
- Детектировать сбои оборудования по характеру телеметрии.

Векторная шаблонная детекция работает исключительно по телеметрии скорости и веса — данным, которые доступны от бортовых датчиков (тензометрия подвески, одометр) даже без спутниковой навигации.[24][25]

По сравнению с нейросетевыми методами

Исследования показывают, что Bi-LSTM с отбором признаков (RFFS) достигает точности 91.75% на задаче распознавания рабочих циклов LHD-машин. Однако эти методы требуют:[19]

- Большого размеченного датасета (тысячи циклов с метками фаз).
- Длительного обучения и подбора гиперпараметров.
- Значительных вычислительных ресурсов для инференса.

Шаблонный подход требует лишь одного размеченного примера каждого типа рейса и работает в реальном времени на обычном сервере. Обнаруженные рейсы в системе показывают совпадения 88–97%, что сопоставимо с нейросетевыми методами.[22]

По сравнению с DTW

Dynamic Time Warping — «золотой стандарт» для сравнения временных рядов разной длины и с временными сдвигами. Но его квадратичная сложность ($O(n \cdot m)$) делает его плохо пригодным для потоковой обработки. Система решает проблему иначе: нормализация к фиксированному размеру вектора и

прямое скалярное произведение, что даёт линейную сложность.[18][17]

Ограничения и перспективы развития

Текущие ограничения

- **Фиксированное окно.** Система не поддерживает DTW-подобное «растяжение» во времени. Если реальный рейс длится значительно дольше или короче шаблона, совпадение упадёт. Частично это решается наличием нескольких шаблонов разной длительности.
- **Два канала данных.** Используются только скорость и вес. Добавление давления масла, оборотов двигателя, температуры и других параметров CAN-шины могло бы повысить точность.[26][19]
- **Отсутствие адаптивности.** Шаблоны статичны; система не обучается на новых данных автоматически. Для добавления нового типа рейса требуется ручная разметка.

Направления развития

- **Автокластеризация** обнаруженных рейсов для выявления новых паттернов без ручной разметки (GMM, DBSCAN).
- **Мультимасштабное окно** — параллельное сравнение с окнами разного размера для устойчивости к вариациям длительности.
- **Интеграция с GPS-данными** — гибрид: первичная детекция по телеметрии, подтверждение по геозонам.
- **Edge-computing** — вынос алгоритма на бортовой контроллер самосвала для автономной работы без связи с сервером.

Позиционирование в отрасли

Традиционные промышленные системы (DISPATCH, Wenco, российская «Карьер», «АвтоГРАФ») решают задачу учёта рейсов через GPS-геозоны и датчики загрузки. Академическое сообщество предлагает нейросетевые подходы (Bi-LSTM,

GMM+MLP) с высокой точностью, но сложным внедрением. Исследование в области потоковой обработки GPS-данных самосвалов на платформе Flink достигло пропускной способности 25 000–66 000 записей/сек для идентификации состояний движения.[20][21][1][19][2][24]

Рассмотренная система занимает промежуточную нишу: она проще нейросетей, информативнее GPS-зон и работает в реальном времени. Подход **шаблонной векторизации** особенно ценен тем, что позволяет кодифицировать экспертное знание диспетчеров в форме конкретных шаблонов — каждый из которых описывает не просто «рейс», а конкретную производственную ситуацию.

Заключение

Подход к детекции рейсов карьерных самосвалов через шаблонную векторизацию телеметрии — практичное инженерное решение, балансирующее между простотой и функциональностью. Его ключевые принципы:

- **Фазовая модель рейса** (погрузка → транспортировка → разгрузка → возврат) создаёт характерный двумерный профиль скорость–вес.
- **Скользящее окно** обеспечивает потоковую обработку телеметрии без хранения полной истории.
- **Нормализованная конкатенация** скоростных и весовых данных в единый вектор позволяет унифицировать сравнение.
- **Косинусное сходство** даёт быструю, масштабо-инвариантную и интерпретируемую метрику совпадения.
- **Библиотека шаблонов** кодифицирует экспертное знание, позволяя различать не только штатные рейсы, но и аномалии, простои, сбои оборудования.

Реализация на стеке Go + React + PostgreSQL + WebSocket обеспечивает производительность, достаточную для обработки потока в реальном времени, с визуализацией и управлением через веб-интерфейс. Подход применим как в составе

полноценной АСУ ГТК, так и как автономный модуль аналитики для отдельного самосвала или группы машин.

References

1. GPS-мониторинг самосвалов БелАЗ, KOMATSU - INNER - Диспетчер видит на экране интерактивную карту карьера со всеми машинами, может назначать рейсы от эк...
2. Горно-диспетчерская система — управление автопарком карьера - На основе GPS-данных система автоматически определяет производственные циклы работы карьерного самос...
3. [PDF] Mining Big Time-series Data on the Web Similarity search, pattern ... - • Euclidean distance is closely related to. – cosine similarity. – dot product. – 'cross-correlation...
4. [PDF] A New Time Series Similarity Measure and Its Smart Grid Applications - Abstract—This paper proposes a novel time-series similarity metric, called Flexibility Distance (FD)...
5. [PDF] Optimization of cycle time for loading and hauling trucks in open-pit ... - Total actual cycle time is equal to loading time plus total hauling time when loaded plus the total ...
6. [PDF] An Agent-based System for Truck Dispatching in Open-pit Mines - Figure 1 shows all the activ- ities that a truck must perform to transport materials from a loading ...
7. A trajectory similarity computation method based on GAT ... - Nature - This algorithm can calculate similarity for long trajectories. It treats long trajectory as a long s...
8. Distance based pattern matching using artificial neural network - In order to overcome these challenges, a distance based pattern-matching approach is used to identif...
9. Pattern Matching Time Series Data with Vector Databases - YouTube - ... vector embeddings by dividing the time series data into overlapping windows and normalizing the ...
10. Template Matching - an overview | ScienceDirect Topics - Template matching is a method for detecting and recognizing objects by moving the template of an ide...
11. Real-Time Adaptive Anomaly Detection in Industrial IoT Environments - ASTREAM incorporates sliding windows, concept drift detection, and updating models in a hashing-base...

12. What is a sliding window approach in time series forecasting? - Milvus - The sliding window approach in time series forecasting is a technique that structures sequential dat...
13. A Guide to Cosine Similarity | Tiger Data - Cosine similarity calculates the cosine of the angle between two vectors, revealing how closely the ...
14. What Is Cosine Similarity? | IBM - At its core, cosine similarity measures how aligned two vectors are by calculating the cosine of the...
15. How to Build Cosine Similarity - OneUptime - Learn to implement cosine similarity for vector search with normalization, batch computation, and op...
16. [PDF] A Shape-based Similarity Measure for Time Series Data with ... - AMSS treats a time series as a vector sequence to focus on the shape of the data and compares data s...
17. Similarity measure of time series based on Angle-distance ... - PMC - Dynamic Time Warping (DTW) is a widely used method for time series similarity measurement, yet its r...
18. Time series classification using Dynamic Time Warping - The combination of DTW with KNN is pretty effective for time series classification. There is a cavea...
19. The load-haul-dump operation cycle recognition based on multi ... - A random forest feature selection (RFFS) and bidirectional long short-term memory (Bi-LSTM) based op...
20. Predicting Haul Truck Travel Times in Underground Mines - PMC - Accurately predicting haul truck (HT) travel times (TT) in underground mines is essential for enhanc...
21. [PDF] Real-Time Driving State Identification and Collision Risk Detection ... - This paper designs and implements an efficient real-time computation system for the massive stream o...
22. US6026341A - Method for detecting truck dump occurrences ... - The method includes determining an initial elevation of a point on the machine. The elevation of the...
23. Shape-based Comparison of Time-series with Ordered Local Trends - DTW+S with Euclidean (default) and cosine distance produces reasonable clustering. All other measure...
24. Контроль загрузки самосвалов - СпутникСервис - Контроль работу карьерных самосвалов в режиме реального времени. Учет количества выполненных рейсов ...

25. [PDF] 18 УДК 622.684 СИСТЕМЫ КОНТРОЛЯ ЗАГРУЗКИ ... - Идея автоматического контроля загрузки карьерных автосамо- свалов появилась более 30 лет назад, но д...
26. Анализ показаний телеметрии при эксплуатации карьерных ... - В статье проводится анализ показаний телеметрии при эксплуатации карьерных самосвалов грузоподъемнос...